

Messages > Tools

Tool use with Claude

Connect Claude to external tools and APIs. See where tools execute, when Claude calls them, and which tool fits your task.

 Copy page 

Tool use (also called function calling) lets Claude call functions that you define or that Anthropic provides. Claude determines when to call a tool based on the user's request and the tool's description. It then returns a structured call that your application executes (client tools) or that Anthropic executes (server tools).

Here's a minimal example using a server tool, the [Web search tool](#), which Anthropic executes for you:

cURL CLI **Python** TypeScript C# Go Java PHP Ruby 

```
client = anthropic.Anthropic()
response = client.messages.create(
    model="claude-opus-5",
    max_tokens=1024,
    tools=[{"type": "web_search_20260209", "name": "web_search"}],
    messages=[{"role": "user", "content": "What's the latest on the Mars
rover?"}],
)
print(response.content)
```

Claude runs the search on Anthropic's infrastructure and returns the cited results in the same response. To have Claude call a function that you define, pass a tool with an `input_schema`, then execute the call when Claude returns a `tool_use` block. [How tool use works](#) shows that round trip end to end. Learn more about [defining tools](#) and [handling tool calls](#).

How tool use works

Tools differ primarily by where the code executes. **Client tools** (including user-defined tools and tools with Anthropic-defined schemas, such as `bash` and `text_editor`) run in your application. Claude responds with `stop_reason: "tool_use"` and one or more `tool_use` blocks. Your code executes the operation and sends back a `tool_result`. **Server tools** (such as `web_search`, `web_fetch`, `code_execution`, and `tool_search`) run on Anthropic's infrastructure: you see the results directly without handling execution, unless Claude calls the tool in the same group of parallel tool calls as one of your client tools (see [Stop reasons and fallback](#)).

reply with the answer.

cURL

CLI

Python

TypeScript

C#

Go

Java

PHP

Ruby




```
{
    "name": "get_weather",
    "description": "Get the current weather for a given location.",
    "input_schema": {
        "type": "object",
        "properties": {
            "location": {
                "type": "string",
                "description": "City and state, e.g. San Francisco, CA",
            }
        },
        "required": ["location"],
    },
}

]
messages = [{"role": "user", "content": "What's the weather in San
Francisco?"}]

# Claude replies with a tool_use block naming the tool and its arguments.
response = client.messages.create(
    model="claude-opus-5",
    max_tokens=1024,
    tools=tools,
    # Ask for at most one tool call per turn.
    tool_choice={"type": "auto", "disable_parallel_tool_use": True},
    messages=messages,
)
tool_use = next(block for block in response.content if block.type ==
"tool_use")
print(f"Claude called {tool_use.name} with {json.dumps(tool_use.input)}")

# Run the tool, then send the result back in a tool_result block.
weather = "15 degrees Celsius, partly cloudy" # your weather lookup goes here
messages += [
    {"role": "assistant", "content": response.content},
    {
        "role": "user",
        "content": [
            {"type": "tool_result", "tool_use_id": tool_use.id, "content":
weather}
        ],
    },
]
followup = client.messages.create(
    model="claude-opus-5",
    max_tokens=1024,
    tools=tools,
    tool_choice={"type": "auto", "disable_parallel_tool_use": True},
    messages=messages,
)
```

```
print(final_text.text)
```

Output



```
Claude called get_weather with {"location": "San Francisco, CA"}  
The current weather in San Francisco is 15 degrees Celsius with partly cloudy  
skies.
```

[Handle tool calls](#) covers each step in detail, including result formatting and error signaling; [Parallel tool use](#) covers responses that call several tools at once. To skip writing this round trip yourself, use [Tool Runner](#): the SDKs execute your tools and send the results back automatically.

For the full conceptual model including the agentic loop and when to choose each approach, see [How tool use works](#).

For connecting to Model Context Protocol (MCP) servers, see the [MCP connector](#). For building your own MCP client, see the Model Context Protocol guide to [building an MCP client](#).

When Claude uses tools

With the default `tool_choice` of `{"type": "auto"}`, Claude determines on each turn whether to call a tool or respond directly. It calls a tool when the request maps to that tool's described capability and the answer isn't already in context. It responds directly for stable knowledge, creative tasks, and conversational turns.

This boundary is steerable through your system prompt. If Claude isn't calling tools when you expect, a light instruction such as `"Use the tools to investigate before responding."` increases tool use. A stronger form such as `"Always call a tool first before responding."` pushes further. Conversely, `"Use your judgment about whether to call a tool or respond directly."` keeps triggering behavior conservative.

To require a tool call rather than rely on prompting, set `tool_choice`.



Guarantee schema conformance with strict tool use

Add `strict: true` to your custom tool definitions to ensure Claude's tool calls always match your schema exactly. See [Strict tool use](#).

Each server tool's page describes its own trigger boundary in more detail.

Choose a tool

For `type` strings, versions, and beta headers, see [Tool reference](#).

Your own tools

For tools you define, you write the schema and your application executes each call.



Define tools

Specify tool schemas, write descriptions, and control when Claude calls your tools.



Handle tool calls

Parse `tool_use` blocks, format `tool_result` responses, and handle errors.

Anthropic-schema client tools

Anthropic publishes the schema and trains Claude on it. Your application still executes each call and returns the `tool_result`.



Memory tool

Store and retrieve information across conversations in files you control.



Bash tool

Run shell commands in a persistent session that maintains state.





Computer use tool

Take screenshots and control the mouse and keyboard in a desktop environment.



Browser use tool

Navigate, read, and interact with webpages in your own browser environment.

Server tools

Server tools run on Anthropic's infrastructure, with no handler code in your application. See [Server tools](#) for the mechanics they share.



Web search tool

Search the web for information beyond the knowledge cutoff, with cited sources.



Web fetch tool

Retrieve the full content of specified web pages and PDF documents.



Code execution tool

Run Python and bash code in a sandboxed container to analyze data and generate files.



Advisor tool



Tool search tool

Work with thousands of tools by discovering and loading them on demand.



MCP connector

Connect to remote MCP servers from the Messages API without a separate MCP client.



[Claude Managed Agents](#) provides a built-in toolset that Claude uses autonomously within a session. For that toolset and the Managed Agents way to add custom tools, see its [Tools](#) page.

Pricing

Tool use requests are priced based on:

1. The total number of input tokens sent to the model (including in the `tools` parameter)
2. The number of output tokens generated
3. For server-side tools, additional usage-based pricing (for example, web search charges per search performed)

Client-side tools are priced the same as any other Claude API request, although server-side tools can incur additional charges based on their specific usage.

The additional tokens from tool use come from:

- The `tools` parameter in API requests (tool names, descriptions, and schemas)
- `tool_use` content blocks in API requests and responses
- `tool_result` content blocks in API requests

When you use `tools`, the API also automatically includes a special system prompt for the model that enables tool use. The number of tool use tokens required for each model is listed in the following table (excluding the additional tokens listed earlier). Note that the table assumes at least 1 tool is provided. If no `tools` are provided, then a tool choice of `none` uses 0 additional system prompt tokens.

Claude Platform Docs

Claude Opus 5	auto , none	286 tokens
	any , tool	406 tokens
Claude Opus 4.8	auto , none	290 tokens
	any , tool	410 tokens
Claude Opus 4.7	auto , none	675 tokens
	any , tool	804 tokens
Claude Opus 4.6	auto , none	497 tokens
	any , tool	589 tokens
Claude Opus 4.5	auto , none	496 tokens
	any , tool	588 tokens
Claude Opus 4.1 (<u>retired, except on Bedrock and Google Cloud</u>)	auto , none	313 tokens
	any , tool	315 tokens
Claude Opus 4 (<u>retired, except on Google Cloud</u>)	auto , none	313 tokens
	any , tool	315 tokens
Claude Sonnet 5	auto , none	354 tokens
	any , tool	474 tokens

Claude Sonnet 4.6	auto , none	497 tokens
	any , tool	589 tokens
Claude Sonnet 4.5	auto , none	496 tokens
	any , tool	588 tokens
Claude Sonnet 4 (<u>retired, except on Bedrock and Google Cloud</u>)	auto , none	313 tokens
	any , tool	315 tokens
Claude Haiku 4.5	auto , none	496 tokens
	any , tool	588 tokens
Claude Haiku 3.5 (<u>retired, except on Bedrock and Google Cloud</u>)	auto , none	264 tokens
	any , tool	355 tokens

These token counts are added to your normal input and output tokens to calculate the total cost of a request.

See the [Models overview](#) table for current per-model prices.

When you send a tool use prompt, like any other API request, the response includes both input and output token counts in the reported `usage` metrics.

Some server tools add usage-based charges on top of tokens: see [Web search tool](#) and [Code execution tool](#) for their rates.

Next steps



How tool use works



Tutorial: Build a tool-using agent

A guided walkthrough from a single tool call to a production-ready agentic loop.



Tool reference

Directory of Anthropic-provided tools and reference for optional tool definition properties.

Cette page vous a-t-elle été utile ?



Code modernization

Coding

Customer support

Financial services

Government

Higher education

K-12 teachers

Life sciences

Partners

Claude on AWS

Claude on Google Cloud

Help and security

Availability

Status

Support

Discord

Terms and policies

Privacy policy

Responsible disclosure policy

Terms of service: Commercial

Terms of service: Consumer

Usage policy

Courses

Use cases

Connectors

Customer stories

Engineering at Anthropic

Events

Powered by Claude

Service partners

Startups program

Company

Anthropic

Careers

Economic Futures

Research

News

Responsible Scaling Policy

Security and compliance

Transparency